

Dashboard Backend API Requirements

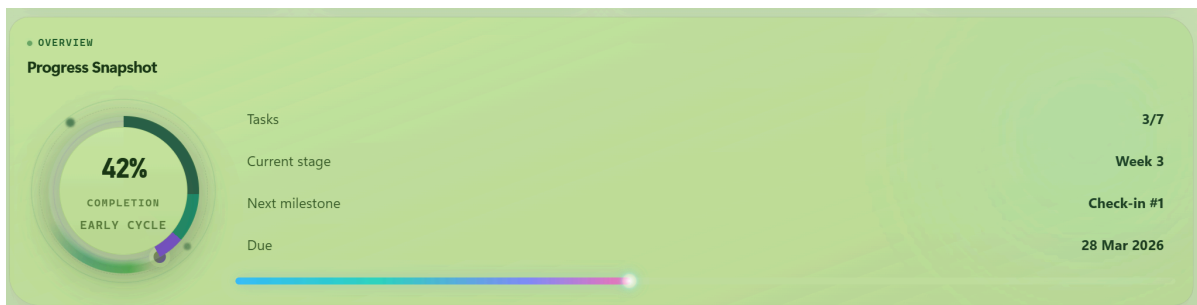
Last updated: 2026-04-21

Audience: backend team.

Purpose: this document lists only the backend API gaps that still block the current `Dashboard Page` from being fully driven by live backend data. Existing APIs that can already be used normally are not the focus of this document.

Note: if any API changes or additions are difficult to implement, please communicate with us at any time. We can further discuss alternative solutions.

1. Dashboard Progress Snapshot API



Why This Is Needed

The `Progress Snapshot` module displays the user's current progress based on values returned by the backend.

Current frontend workaround:

```
GET /tasks/api/v1/tasks/?page_size=100&deleted=False
GET /tasks/api/v1/milestones/?page_size=100
```

This is only an approximation because:

- `Tasks` has no `completed` or `status` field.
- `Tasks` does not expose task assignees in the list response.
- `Milestone` has `completed`, but does not expose due date, start date, end date, or ordering.
- The frontend has to guess progress from milestones and related tasks.
- The Dashboard needs role-aware/user-aware progress, not a global task list.

Recommended Endpoint

```
GET /dashboard/v1/progress/
GET /dashboard/v1/progress/?group_id=12
```

The endpoint should infer the current user from authentication.

For students, return progress for their active group or current program cycle.

For mentors/supervisors, return progress for assigned groups, or support `group_id`.

Recommended Response

```
{
  "completion_rate": 42,
  "completed_tasks": 3,
  "total_tasks": 7,
  "current_stage": "Week 3",
  "next_milestone": {
    "id": 21,
    "name": "Check-in #1",
    "due_date": "2026-03-28T00:00:00Z"
  },
  "scope": {
    "type": "user",
    "user_id": 7,
    "group_id": 12,
    "track_id": 1
  },
  "updated_at": "2026-04-20T08:30:00Z"
}
```

Field Mapping

Dashboard field	Backend field
completionRate	completion_rate
completedTasks	completed_tasks
totalTasks	total_tasks
currentweek	current_stage
nextMilestone	next_milestone.name
nextMilestoneDate	next_milestone.due_date

Alternative: Enhance Existing Tasks APIs

If a Dashboard-specific endpoint is not desired, please enhance existing task/milestone APIs:

```
GET /tasks/api/v1/tasks/?assigned_to=me&deleted=False&page_size=100
GET /tasks/api/v1/tasks/?group_id=12&deleted=False&page_size=100
GET /tasks/api/v1/milestones/?group_id=12&page_size=100
```

Recommended task list item:

```
{
  "id": 101,
  "task_name": "Submit draft project plan",
  "task_description": "Upload the first project plan draft.",
  "due_date": "2026-03-28T00:00:00Z",
  "deleted_flag": false,
}
```

```

"completed": false,
"status": "todo",
"milestone": 21,
"group": 12,
"assignees": [
  {
    "id": 7,
    "first_name": "Jamie",
    "last_name": "Chen",
    "email": "jamie@example.org"
  }
]
}

```

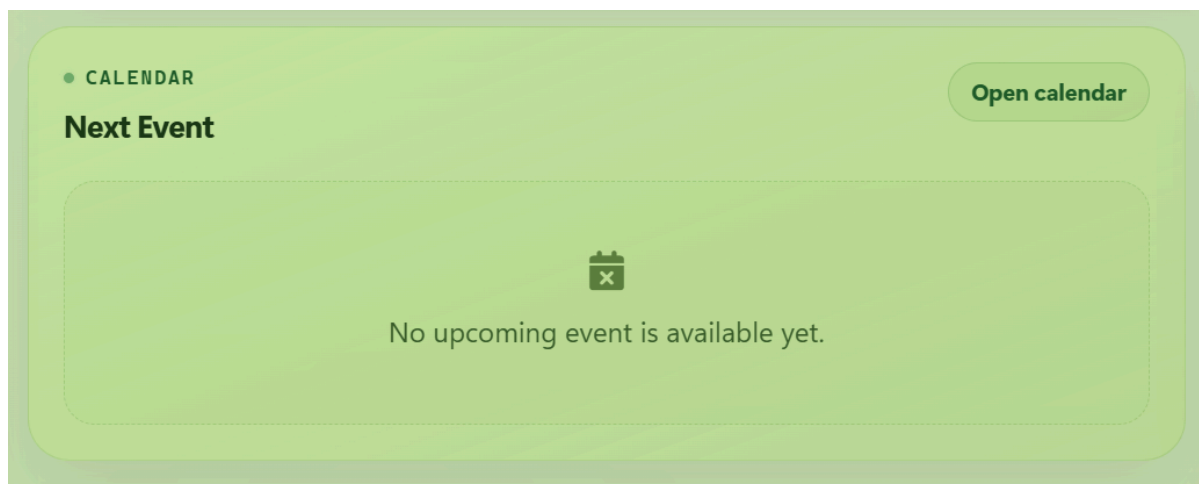
Recommended milestone list item:

```

{
  "id": 21,
  "group": 12,
  "milestone_name": "Check-in #1",
  "completed": false,
  "deleted_flag": false,
  "start_date": "2026-03-21T00:00:00Z",
  "due_date": "2026-03-28T00:00:00Z",
  "sort_order": 30
}

```

2. Personalized Next Event API



Why This Is Needed

The `Next Event` module currently uses:

```
GET /events/v1/?page_size=10
```

This endpoint can return upcoming events, but the Dashboard needs the next event relevant to the current user:

- Student: event for their track/group or event they are invited to.
- Mentor/supervisor: event for assigned groups or invited events.

- Admin: next event within admin track scope or platform-wide.

Without this capability, the frontend may show a valid event, but it may not be the best next event for the current user.

Recommended Endpoint Option A

```
GET /dashboard/v1/next-event/
```

Recommended response:

```
{
  "id": 101,
  "event_name": "Group Check-in Session",
  "description": "Mentor-led group check-in.",
  "track": 1,
  "group": 12,
  "event_type": "workshop",
  "start_datetime": "2026-04-28T06:00:00Z",
  "ends_datetime": "2026-04-28T07:00:00Z",
  "location": null,
  "humanitix_link": "https://example.org/event",
  "event_image": "https://example.org/event.jpg",
  "is_virtual": true,
  "rsvp_status": "pending"
}
```

Return `null` or HTTP `204` if there is no relevant upcoming event.

Recommended Endpoint Option B

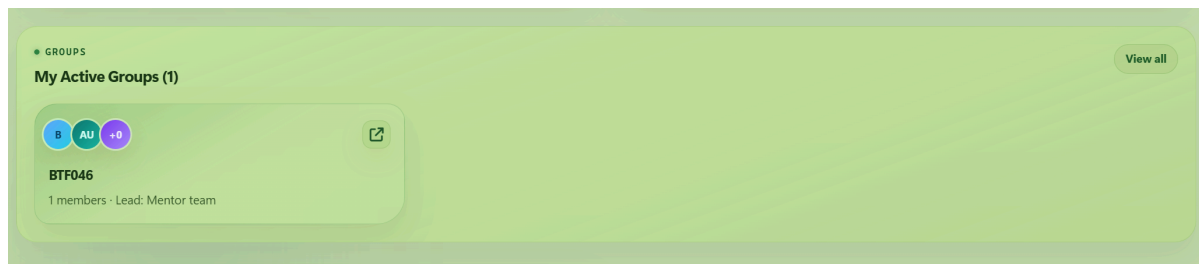
Enhance the existing events list API:

```
GET /events/v1/?audience=me&page_size=1&ordering=start_datetime
```

Recommended query parameters:

Parameter	Purpose
<code>audience=me</code>	Limit to events relevant to the current user.
<code>track_id</code>	Optional admin/mentor filtering.
<code>group_id</code>	Optional group-specific filtering.
<code>ordering=start_datetime</code>	Ensure the earliest starting event appears first.

3. Group Preview Enhancements



Why This Is Needed

The `Active Groups` module currently combines:

```
GET /groups/groups/?page_size=20
GET /groups/group-members/?page_size=100
GET /groups/tracks/?page_size=100
```

This works, but it requires multiple requests and still cannot show a clean mentor/lead name because `group-members` only returns the user id.

Recommended Endpoint Option A

Enhance the existing group list response:

```
GET /groups/groups/?mine=true&page_size=20
```

Recommended list item:

```
{
  "id": 12,
  "group_name": "BTF046",
  "track": 1,
  "track_name": "AUS-NSW",
  "created_at": "2026-03-01T00:00:00Z",
  "deleted_at": null,
  "member_count": 4,
  "lead_user": {
    "id": 3,
    "first_name": "Anita",
    "last_name": "Pickard",
    "email": "anita@example.org",
    "role": "Mentor"
  }
}
```

Recommended Endpoint Option B

Create a Dashboard-specific group preview endpoint:

```
GET /dashboard/v1/groups-preview/
GET /dashboard/v1/groups-preview/?track_id=1&page_size=4
```

Recommended response:

```
{
  "count": 3,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 12,
      "name": "BTF046",
      "track_id": 1,
      "track_name": "AUS-NSW",
      "member_count": 4,
      "lead_name": "Anita Pickard",
      "status": "active"
    }
  ]
}
```

Minimum Enhancements Needed

Field / query	Why
<code>mine=true</code>	Lets non-admin Dashboard request only the current user's groups.
<code>track_name</code>	Avoids a second <code>/groups/tracks/</code> request.
<code>member_count</code>	Avoids counting memberships on the frontend.
<code>lead_user</code> or <code>lead_name</code>	Lets Dashboard display mentor/lead instead of <code>Mentor #id</code> .

4. Existing Task API Boolean Query Handling

Why This Is Needed

The frontend currently uses:

```
GET /tasks/api/v1/tasks/?page_size=100&deleted=False
```

This casing is intentional because the current backend returns a 500 error for lowercase `deleted=false`.

The API should accept common boolean query formats and should not return 500 because of query parameter casing.

Recommended Backend Behavior

Supported values:

Query value	Meaning
<code>true</code> , <code>True</code> , <code>1</code> , <code>yes</code>	<code>deleted_flag=True</code>
<code>false</code> , <code>False</code> , <code>0</code> , <code>no</code>	<code>deleted_flag=False</code>

Invalid values should return HTTP `400`, not HTTP `500`.

Recommended error response:

```
{
  "detail": "Invalid deleted value. Expected true or false."
}
```

Final Required API Gap Summary

Required for the current Dashboard to be fully driven by live backend data:

1. `GET /dashboard/v1/progress/` or equivalent task/milestone API enhancements.
2. `GET /dashboard/v1/next-event/` or `GET /events/v1/?audience=me&page_size=1&ordering=start_datetime`.
3. Group preview enhancements: `mine=true`, `track_name`, `member_count`, and `lead_user` / `lead_name`.
4. Robust boolean parsing for `GET /tasks/api/v1/tasks/?deleted=...`.